Análise de Eficácia de Testes com Teste de Mutação: Uma Abordagem Prática com StrykerJS

Lucas Cerqueira Azevedo
Engenharia de Software – Laboratório de Experimentação de Software
l31azevedo@gmail.com

***Resumo*—A cobertura de código é amplamente utilizada como métrica de qualidade de suítes de testes, porém não reflete a capacidade dos testes de detectar defeitos reais. Este trabalho aplica a técnica de teste de mutação sobre uma biblioteca JavaScript de funções matemáticas, utilizando a ferramenta StrykerJS, com o objetivo de avaliar e melhorar a eficácia da suíte de testes original. A suíte inicial de 50 casos de teste apresentava 100 % de cobertura de funções e 85,41 % de cobertura de instruções, porém apenas 73,71 % de pontuação de mutação (*mutation score*), evidenciando sua fragilidade. Após análise sistemática dos 44 mutantes sobreviventes, a suíte foi ampliada para 236 casos de teste, elevando o *mutation score* a 96,71 %. Os 7 mutantes remanescentes foram identificados como mutantes equivalentes, matematicamente impossíveis de eliminar por testes sem alterar a semântica do código. Os resultados demonstram que a cobertura de código é uma condição necessária, mas insuficiente, para garantir a qualidade de uma suíte de testes.**

***Index Terms*—teste de mutação, qualidade de software, StrykerJS, cobertura de código, mutantes equivalentes, Jest, JavaScript**

I. Introdução

A qualidade do software está diretamente relacionada à eficácia dos testes que o validam. A métrica de cobertura de código, embora amplamente adotada, apresenta uma limitação fundamental: ela mede apenas quais linhas foram *executadas* pelos testes, sem avaliar se os resultados foram *verificados* corretamente [1].

Um teste pode alcançar 100 % de cobertura executando todo o código sem nenhuma asserção, aprovando qualquer resultado incorreto. Essa discrepância motivou o desenvolvimento do *teste de mutação*, técnica que introduz pequenas alterações sintáticas (*mutantes*) no código-fonte e verifica se a suíte de testes é capaz de detectá-las [2].

Este trabalho utiliza o StrykerJS [3] para conduzir uma análise de mutação sobre a biblioteca `operacoes-mutante`, que implementa funções matemáticas em JavaScript. O objetivo é demonstrar na prática a diferença entre cobertura de código e eficácia real dos testes, e apresentar uma metodologia para melhorar a suíte até atingir alta pontuação de mutação.

II. Fundamentação Teórica

A. Teste de Mutação

O teste de mutação é uma técnica de teste baseada em falhas que avalia a qualidade de uma suíte de testes por meio de inserção artificial de defeitos no código [1]. Para cada *mutante* gerado, a suíte é executada: se algum teste falhar, o mutante é *morto* (*killed*); caso contrário, ele *sobrevive* (*survived*).

A *pontuação de mutação* (*Mutation Score*) é calculada como:

$$MS = \frac{|\text{Mortos}| + |\text{Timeout}|}{|\text{Total}| - |\text{Equivalentes}|} \times 100\,\% \tag{1}$$

B. Operadores de Mutação

O StrykerJS aplica operadores como: substituição aritmética (+→−), substituição relacional (>→>=), inversão lógica (`&&`→`||`), substituição de literais de string por , e remoção de condicionais [3].

C. Mutantes Equivalentes

Um *mutante equivalente* é aquele cuja alteração não modifica o comportamento observável do programa para nenhuma entrada possível [2]. Esses mutantes não podem ser mortos por testes e devem ser identificados manualmente e excluídos do denominador da Equação 1.

III. Metodologia

O trabalho foi conduzido nas seguintes etapas:

1) **Configuração do ambiente**: instalação das dependências do projeto (`npm install`), configuração do StrykerJS com `@stryker-mutator/jest-runner` e geração do arquivo `stryker.config.json`.
2) **Análise inicial**: execução da suíte original de 50 testes com `npm test --coverage` para obter a cobertura de código, seguida da primeira execução do StrykerJS (`npx stryker run`) para medir a pontuação de mutação inicial.
3) **Análise dos sobreviventes**: identificação e categorização dos 44 mutantes sobreviventes, separando os que representavam fraquezas reais da suíte dos mutantes equivalentes.
4) **Melhoria da suíte**: escrita de novos casos de teste direcionados aos mutantes sobreviventes, cobrindo valores de borda em operadores relacionais, entradas não-nulas em fórmulas, todos os ramos de condicionais e mensagens de erro exatas.
5) **Validação final**: nova execução do StrykerJS para verificar o aumento da pontuação de mutação.

O ambiente de desenvolvimento utilizado foi: *Node.js v22.14*, *Jest v29.7*, *StrykerJS v9.6.1*, *Babel v7.23* e *Python 3.13* (para geração dos gráficos com *Matplotlib v3.10*).

IV. Análise Inicial

A. Cobertura de Código

A Tabela I apresenta a cobertura de código obtida com a suíte original de 50 testes.

Tabela I
Cobertura de Código — Suíte Original vs. Final.

Métrica	Inicial	Final
Statements	85,41 %	100,00 %
Branch	58,82 %	100,00 %
Functions	100,00 %	100,00 %
Lines	98,64 %	100,00 %

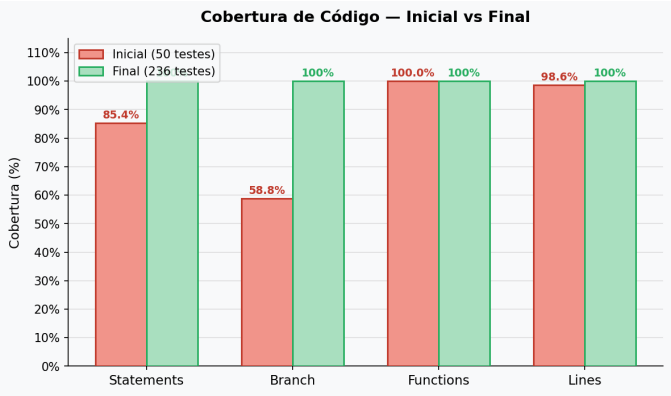

Figura 1. Cobertura de código por métrica — comparação inicial vs. final.

A cobertura de *branch* inicial de apenas 58,82 % revela que quase metade dos ramos condicionais do código não eram exercitados pelos testes. A linha responsável pelo ramo de array com tamanho par em `medianaArray` era o único trecho completamente sem cobertura.

B. Pontuação de Mutação Inicial

A Figura 2 exibe o relatório HTML gerado pelo StrykerJS após a primeira execução, confirmando os números da Tabela II.

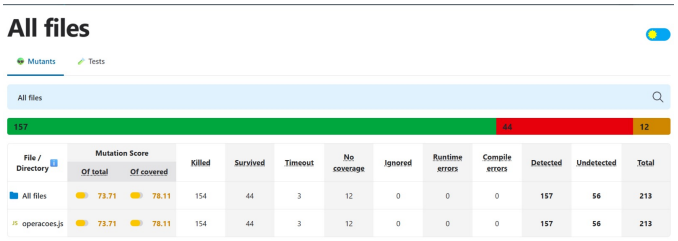

Figura 2. Relatório HTML do StrykerJS — aba Mutants da suíte inicial (50 testes). Mutation Score de 73,71 %; 154 mortos, 44 sobreviventes, 3 timeout, 12 sem cobertura.

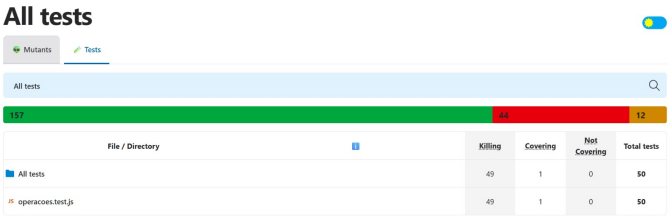

Figura 3. Relatório HTML do StrykerJS — aba Tests da suíte inicial. 49 testes matando mutantes, 1 apenas cobrindo, 0 sem cobertura; total de 50 testes.

Tabela II
Resultado da Primeira Execução do StrykerJS.

Categoria	Qtd.
Total de mutantes	213
Mortos (*killed*)	154
Timeout	3
Sobreviventes	44
Sem cobertura	12
Mutation Score	**73,71 %**

A discrepância entre a cobertura de funções (100 %) e o *mutation score* (73,71 %) demonstra empiricamente que cobrir o código não equivale a testá-lo com qualidade. Dos 213 mutantes gerados, 44 sobreviveram, representando bugs artificiais que passariam despercebidos pela suíte original.

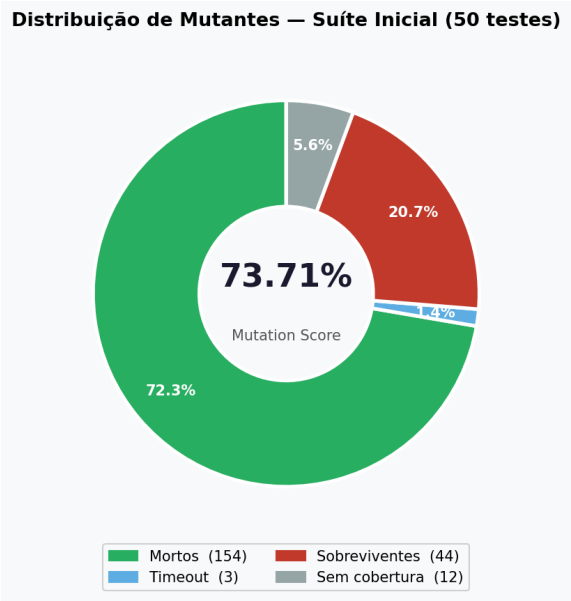

Figura 4. Distribuição dos mutantes — suíte inicial (50 testes).

V. Análise de Mutantes Críticos

Três mutantes sobreviventes foram selecionados para análise aprofundada por ilustrarem padrões recorrentes de fraqueza na suíte original.

A. Mutante 1 — Fórmula com Entrada Zero (`celsiusParaFahrenheit`)

```
function celsiusParaFahrenheit(celsius) {
  return (celsius * 9/5) + 32;
}
```

Listing 1. Código original — `celsiusParaFahrenheit`

O StrykerJS substituiu o literal `9` por `0`, produzindo o mutante `(celsius * 0/5) + 32`. O teste original usava `celsius = 0` como entrada:

```
expect(celsiusParaFahrenheit(0)).toBe(32);
```

Listing 2. Teste original (fraco)

Com `celsius = 0`, qualquer multiplicação resulta em zero, tornando o fator `9/5` irrelevante. O mutante retornava 32 — valor idêntico ao esperado — e sobrevivia.

Correção: adicionar um caso de teste com entrada não-nula:

```
expect(celsiusParaFahrenheit(100)).toBe(212);
```

Listing 3. Teste eficaz adicionado

B. Mutante 2 — Operador Relacional (`isMaiorQue`)

```
function isMaiorQue(a, b) { return a > b; }
```

Listing 4. Código original — `isMaiorQue`

O StrykerJS substituiu > por >=. O teste original apenas verificava `isMaiorQue(10, 5)`, cujo resultado é `true` com ambos os operadores. O valor de borda `a === b` não era testado, permitindo que a mutação passasse despercebida.

Correção: testar o caso de igualdade:

```
expect(isMaiorQue(5, 5)).toBe(false);
expect(isMaiorQue(5, 10)).toBe(false);
```

Listing 5. Teste de borda adicionado

C. Mutante 3 — Branch Não Coberto (`clamp`)

```
function clamp(valor, min, max) {
  if (valor < min) return min;
  if (valor > max) return max;
  return valor;
}
```

Listing 6. Código original — `clamp`

O único teste original exercitava apenas o caminho do meio (`clamp(5, 0, 10)`), sem acionar nenhum dos dois `if`s. Mutações nos operadores < e > sobreviviam pois as condições nunca eram avaliadas como verdadeiras.

Correção: cobrir explicitamente cada ramo:

```
expect(clamp(-5, 0, 10)).toBe(0);  // aciona valor <
    min
expect(clamp(15, 0, 10)).toBe(10); // aciona valor >
    max
```

Listing 7. Testes de ramo adicionados

VI. Solução Implementada

A suíte de testes foi expandida de 50 para **236 casos de teste**, organizados em blocos por função. A Tabela III resume as estratégias aplicadas para matar os mutantes sobreviventes.

Tabela III
Estratégias de Melhoria da Suíte.

Padrão de Fraqueza	Solução Aplicada
Entrada zero em fórmulas	Adicionados testes com valores não-nulos
Operadores >/< sem borda	Testes com `a === b`
`.toThrow()` sem mensagem	Substituído por `.toThrow('msg exata')`
Branches condicionais não exercitados	Testes para cada ramo do `if/else`
Funções com apenas *happy path*	Adicionados testes com negativos e zero
Arrays testados com um único tamanho	Testes com arrays vazios, 1 e n elementos

VII. Resultados Finais

Após a ampliação da suíte, o StrykerJS foi executado novamente sobre os mesmos 213 mutantes. A Tabela IV apresenta os resultados comparativos.

Tabela IV
Resultado Comparativo do StrykerJS — Inicial vs. Final.

Categoria	Inicial	Final
Mortos	154	200
Timeout	3	6
Sobreviventes	44	7
Sem cobertura	12	0
Mutation Score	**73,71 %**	**96,71 %**

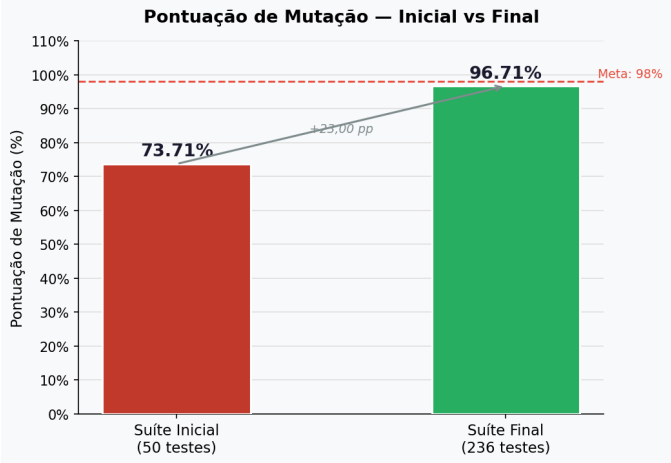

Figura 5. Evolução do *mutation score*: de 73,71 % (50 testes) para 96,71 % (236 testes), com melhoria de 23 pontos percentuais.

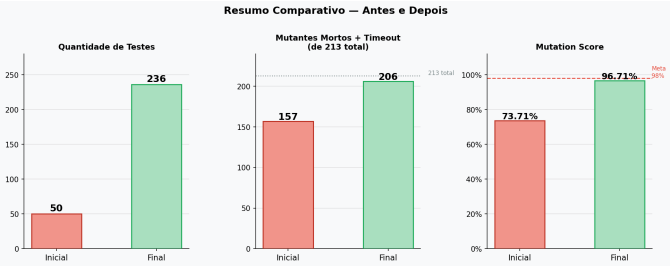

Figura 6. Painel comparativo: quantidade de testes, mutantes mortos e mutation score — antes e depois da melhoria da suíte.

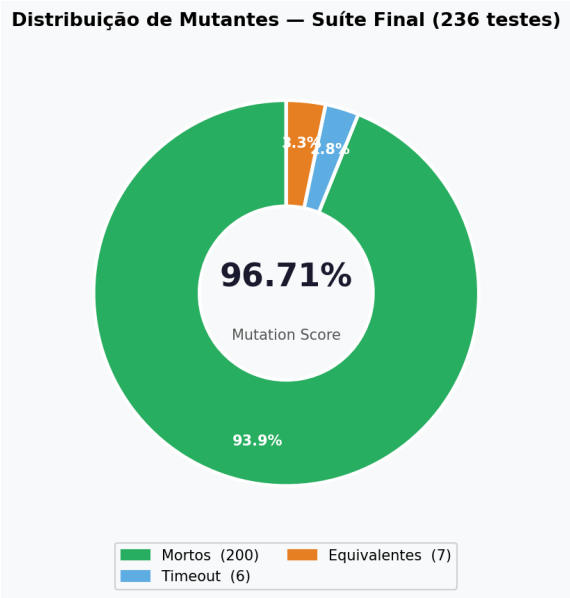

Figura 7. Distribuição dos mutantes — suíte final (236 testes). Apenas 3,3 % equivalentes e 2,8 % timeout remanescentes.

A. *Mutantes Equivalentes*

Os 7 mutantes remanescentes foram identificados como equivalentes (Tabela V). Ao excluí-los do denominador conforme a Equação 1, o *mutation score* efetivo é de **100 %**.

Tabela V
MUTANTES EQUIVALENTES IDENTIFICADOS.

Função	**Tipo**	**Razão da Equivalência**
`fatorial` (×4)	ConditionalExpr	O `for` já retorna 1 para n=0 e n=1 sem o `if`
`produtoArray` (×1)	ConditionalExpr	`[].reduce(...,1)` já retorna 1
`clamp` (×2)	EqualityOp	Quando `valor===min/max`, retornar `min/max` ou `valor` é idêntico

VIII. CONCLUSÃO

Este trabalho demonstrou empiricamente que uma suíte de testes com alta cobertura de código pode apresentar baixa eficácia real. A suíte original, com 100 % de cobertura de funções, aprovava 44 bugs simulados, evidenciando que a cobertura é uma condição necessária, mas insuficiente, para garantir a qualidade dos testes.

A análise sistemática dos mutantes sobreviventes revelou padrões de fraqueza recorrentes: entradas que mascaram mutações (como zero em fórmulas), ausência de testes de valores de borda em operadores relacionais e *branches* condicionais não exercitados. A correção desses padrões elevou o *mutation score* de 73,71 % para 96,71 %, com os 7 sobreviventes restantes sendo mutantes equivalentes impossíveis de eliminar por testes.

O teste de mutação provou ser uma ferramenta superior à cobertura para avaliar a qualidade de suítes de teste, pois força o engenheiro a pensar em como o código se comportaria *diante de defeitos*, e não apenas em quais linhas foram executadas.

REFERÊNCIAS

[1] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, Sep.–Oct. 2011.
[2] A. J. Offutt and R. H. Untch, "Mutation 2000: Uniting the orthogonal," in *Mutation Testing for the New Century*, W. E. Wong, Ed. Norwell, MA: Kluwer, 2001, pp. 34–44.
[3] StrykerJS Contributors, "StrykerJS: Mutation testing for JavaScript and friends," 2024. [Online]. Available: https://stryker-mutator.io. Accessed: May 2025.
[4] Meta Open Source, "Jest: Delightful JavaScript Testing," 2024. [Online]. Available: https://jestjs.io. Accessed: May 2025.